

Assignment 1

Import necessary libraries

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import mean_squared_error, r2_score
```

```
data = pd.read_csv('advertising.csv')
```

```
print(data)
```

Define the independent variable (feature) and dependent variable (target)

```
X = data[['TV']]
```

```
y = data['Sales']
```

Split the data into training and testing sets (80% train, 20% test)

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Initialize and train the Linear Regression model

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

Make predictions on the test set

```
y_pred = model.predict(X_test)
```

Evaluate the model's performance

```
mse = mean_squared_error(y_test, y_pred)

r2 = r2_score(y_test, y_pred)


# Output the evaluation metrics

print(f"Mean Squared Error (MSE): {mse}")

print(f"R-squared (R2) Score: {r2}")


# Visualize the results

plt.scatter(X_test, y_test, color='blue', label='True Values')

plt.plot(X_test, y_pred, color='red', label='Regression Line')

plt.xlabel('TV')

plt.ylabel('Sales')

plt.title('Linear Regressin Model')

plt.legend()

plt.show()
```

Assignment 2

```
import numpy as np
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
df=pd.read_csv('log.csv')
```

```
X=df.iloc[:, :-1]
```

```
y=df.iloc[:, -1]
```

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

```
scaler =StandardScaler()
```

```
X_train =scaler.fit_transform(X_train)
```

```
X_test =scaler.transform(X_test)
```

```
model =LogisticRegression()
```

```
model.fit(X_train,y_train)
```

```
y_pred =model.predict(X_test)
```

```
accuracy =accuracy_score(y_test,y_pred)
```

```
conf_matrix =confusion_matrix(y_test,y_pred)
```

```
report =classification_report(y_test, y_pred)
```

```
print(f'Accuracy : {accuracy: .4f}')
```

```
print('Conf_matrix:')
```

```
print(conf_matrix)
```

```
print('classification Report :')
```

```
print(report)
```

Assignment 3

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix


df=pd.read_csv('log.csv')


X=df.iloc[:, :-1]

y=df.iloc[:, -1]


X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)


scaler =StandardScaler()


X_train =scaler.fit_transform(X_train)


X_test =scaler.transform(X_test)


model =DecisionTreeClassifier()

model.fit(X_train,y_train)
```

```
y_pred =model.predict(X_test)

accuracy =accuracy_score(y_test,y_pred)


conf_matrix =confusion_matrix(y_test,y_pred)

report =classification_report(y_test, y_pred)

print(f'Accuracy : {accuracy: .4f}')


print('Confusion matrix:')

print(conf_matrix)


print('classification Report :')

print(report)


plt.figure(figsize=(15,10))

plot_tree(model ,filled= True, feature_names=df.columns[:-1], class_names=str(np.unique(y)))

plt.show()
```

Assignment 4

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.cluster import KMeans
```

```
from sklearn.datasets import make_blobs
```

```
X,y=make_blobs(n_samples=300, centers=4, random_state=42)
```

```
kmeans= KMeans(n_clusters=4, random_state=42)
```

```
kmeans.fit(X)
```

```
centroids =kmeans.cluster_centers_
```

```
labels =kmeans.labels_
```

```
plt.figure(figsize=(8,6))
```

```
plt.scatter(X[:,0],X[:,1], c=labels, cmap='viridis', marker='o', edgecolor='k')
```

```
plt.scatter(centroids[:,0],centroids[:,1], marker='X', s=200, c='red',label='centroids')
```

```
plt.title('K-means Clustering')
```

```
plt.xlabel('Feature 1')
```

```
plt.ylabel('Feature 2')
```

```
plt.legend()
```

```
plt.show()
```

Assignment 5

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.decomposition import PCA #principle component analysis

from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split


df =pd.read_csv('log.csv')


print(df.head())# print first5 records


X=df.iloc[:, :-1].values

y=df.iloc[:, -1].values


scaler =StandardScaler()

X_scaled= scaler.fit_transform(X)


pca = PCA(n_components=2)

X_pca =pca.fit_transform(X_scaled)


print("Explained variance ratio",pca.explained_variance_ratio_)


plt.figure(figsize=(8,6))

plt.scatter(X_pca[:,0],X_pca[:,1],c=y,cmap='viridis', edgecolor='k',s=100)
```



```
plt.title("PCA of Dataset")  
plt.xlabel("Principal component 1")  
plt.ylabel("Principal component 2")  
plt.colorbar(label='Target Class')  
plt.show()
```

Assignment 6

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import accuracy_score
```

```
from sklearn.preprocessing import LabelEncoder
```

```
df=pd.read_csv('log.csv')
```

```
print(df.head())
```

```
df= df.dropna()
```

```
X=df.iloc[:, :-1].values
```

```
y=df.iloc[:, -1].values
```

```
label_encoder = LabelEncoder()
```

```
y= label_encoder.fit_transform(y)
```

```
X_train, X_test, y_train, y_test= train_test_split(X,y,test_size=0.3, random_state=42)
```

```
rf_classifier =RandomForestClassifier(n_estimators=100, random_state=42)
```

```
rf_classifier.fit(X_train,y_train)
```

```
y_pred =rf_classifier.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print(f'Accuracy of random forest on test data:{accuracy *100:2f}%')
```

Assignment 7

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import AdaBoostClassifier
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.metrics import accuracy_score
```

```
from sklearn.preprocessing import LabelEncoder
```

```
df= pd.read_csv('log.csv')
```

```
print(df.head())
```

```
df= df.dropna()
```

```
X =df.iloc[:, :-1].values
```

```
y =df.iloc[:, -1].values
```

```
label_encoder = LabelEncoder()
```

```
y= label_encoder.fit_transform(y)
```

```
X_train, X_test, y_train, y_test= train_test_split(X,y,test_size=0.3, random_state=42)
```

```
base_estimator = DecisionTreeClassifier(max_depth=1)
```

```
adaboost_classifier = AdaBoostClassifier(base_estimator= base_estimator,  
n_estimators=50,random_state=42)
```

```
adaboost_classifier.fit(X_train, y_train)
```

```
y_pred = adaboost_classifier.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print(f'Accuracy of AdaBoost on test data:{accuracy * 100:.2f}%')
```

Assignment 8

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.svm import SVC
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
df= pd.read_csv('pca.csv')
```

```
print(df.head())
```

```
df= df.dropna()
```

```
X =df.iloc[:, :-1]
```

```
y =df.iloc[:, -1]
```

```
X_train, X_test, y_train, y_test= train_test_split(X,y,test_size=0.2, random_state=42)
```

```
scaler =StandardScaler()
```

```
X_train =scaler.fit_transform(X_train)
```

```
X_test=scaler.transform(X_test)
```

```
svm_model=SVC(kernel='linear', random_state=42)
```

```
svm_model.fit(X_train, y_train)
```

```
y_pred=svm_model.predict(X_test)
```

```
accuracy= accuracy_score(y_test, y_pred)
```

```
conf_matrix=confusion_matrix(y_test, y_pred)
```

```
report= classification_report(y_test, y_pred)
```

```
print(f'Accuracy: {accuracy: .4f}')
```

```
print('Confusion Matrix')
```

```
print(conf_matrix)
```

```
print('Classification Report:')
```

```
print(report)
```

```
if X.shape[1]==2:
```

```
    h=.02
```

```
    x_min, x_max= X_train[:,0].min() -1,X_train[:,0].max() +1
```

```
    y_min, y_max= X_train[:,1].min() -1,X_train[:,1].max() +1
```

```
    xx, yy =np.meshgrid(np.arange(x_min , x_max, h),np.arange(y_min, y_max, h))
```

```
Z=svm_model.predict(np.c_[xx.ravel(),yy.ravel()])
```

```
Z = Z.reshape(xx.shape)
```

```
plt.contourf(xx,yy,Z,alpha=0.8)
```

```
plt.scatter(X_train[:,0],X_train[:,1], c=y_train, edgecolors='k',marker='o',cmap=plt.cm.Paired)
```

```
plt.title('SVM Decision boundary with Linear kernel')
```

```
plt.xlabel('Feature 1')
```

```
plt.ylabel('Feature 2')
```

```
plt.show()
```

Assignment 9

```
import numpy as np

import pandas as pd

import tensorflow as tf

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, Input

import matplotlib.pyplot as plt

df=pd.read_csv('diabetes.csv')

df.head()

print('Number of Rows:', df.shape[0])

print('Number of Columns:', df.shape[1])

print('Number of Patients with outcome 1:', df.Outcome.sum())

print('Event Rate:', round(df.Outcome.mean()*100,2),'%')

df.describe()

from sklearn.model_selection import train_test_split

X=df.to_numpy()[0:8]

y=df.to_numpy[:,8]\

speed=42

X_train, X_test, y_train, y_test= train_test_split(X,y,test_size=0.25, random_state=speed)

print(f'Shape of Train Data:{X_train.shape}')

print(f'Shape of Test Data:{X_test.shape}')

model= sequential([

    Input(sape=(8,)),
```



```
Dense(24, activation='relu'),  
Dense(12, activation='relu'),  
Dense(1, activation='sigmoid'),  
)
```